

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	LOKESH S	Reg. No.:	192572151
Section / Batch:	CSE -AI /2 ND YEAR	Date:	25/08/2026

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system receives interrupt requests from sensors, communication modules, and storage devices at unpredictable intervals. Develop pseudocode for a nested interrupt handling mechanism that allows a higher-priority interrupt to pre-empt a lower-priority service routine. The algorithm should save the current processor context, service the higher-priority interrupt, restore the interrupted routine, and ensure that pending interrupts are handled correctly.
2. A data-intensive embedded system must transfer a large amount of information between memory and multiple I/O devices without continuously occupying the processor. Write pseudocode for a scatter-gather DMA mechanism that transfers data stored in multiple non-contiguous memory locations. The algorithm should maintain a list of memory descriptors, initiate successive transfers, update descriptor status, and handle completion and error conditions.
3. A computer system supports several I/O devices that generate data at different rates, causing temporary variations in the amount of data waiting for processing. Design pseudocode for an I/O buffering mechanism that manages incoming and outgoing data using appropriate buffer allocation and release strategies. The algorithm should monitor buffer occupancy, handle buffer-full and buffer-empty conditions, and prevent data loss while maintaining efficient device communication.
4. A high-speed embedded platform contains multiple communication devices competing for shared system resources. Develop pseudocode for a priority-based resource allocation mechanism that assigns communication resources according to device urgency and bandwidth requirements. The algorithm should dynamically update priorities, allocate resources to eligible devices, release resources after transmission, and prevent low-priority devices from being indefinitely delayed.
5. A real-time system must transfer data from several peripheral devices while meeting strict timing constraints. Write pseudocode for a real-time I/O scheduling mechanism that determines when each device should perform data transfers based on deadlines, transfer duration, priority, and available system bandwidth. The algorithm should schedule time-critical transfers first, monitor missed deadlines, and reschedule pending requests to improve overall system responsiveness.

Code of Conduct

I LOKESH S (192572151) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: LOKESH S

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator

Faculty Incharge

1. Nested Interrupt Handling Mechanism

Goal: allow a higher-priority interrupt to preempt a lower-priority service routine currently executing, by saving the interrupted routine's processor context, servicing the new interrupt, restoring the original context, and then draining any interrupts that arrived and were queued during servicing.

```
STRUCTURE InterruptVector
    priority      : Integer
    ISR_Address   : Pointer
    pending       : Boolean
END STRUCTURE

GLOBAL interruptTable[NUM_SOURCES] // indexed by interrupt ID
GLOBAL currentPriority = LOWEST_PRIORITY
GLOBAL contextStack = EMPTY_STACK
GLOBAL pendingQueue = EMPTY_QUEUE

FUNCTION MasterInterruptHandler(interruptID)
    newPriority = interruptTable[interruptID].priority

    // Step 1: Decide whether to preempt
    IF newPriority <= currentPriority THEN
        // Lower or equal priority: queue it, do not preempt
        ENQUEUE(pendingQueue, interruptID)
        RETURN
    END IF

    // Step 2: Save context of the interrupted routine
    savedContext = SaveProcessorContext() // PC, SP, registers, flags
    PUSH(contextStack, savedContext)
    PUSH(contextStack, currentPriority)

    // Step 3: Raise priority mask so only higher-priority IRQs can preempt
    previousPriority = currentPriority
    currentPriority = newPriority
    SetInterruptMask(currentPriority)
    EnableGlobalInterrupts() // allow further nesting

    // Step 4: Service the higher-priority interrupt
    interruptTable[interruptID].pending = FALSE
    CALL interruptTable[interruptID].ISR_Address()

    // Step 5: Restore the preempted routine
    DisableGlobalInterrupts()
```

```

currentPriority = previousPriority
SetInterruptMask(currentPriority)
restoredContext = POP(contextStack)
RestoreProcessorContext(restoredContext)
EnableGlobalInterrupts()

// Step 6: Service any interrupts queued while this one ran
ProcessPendingQueue()
END FUNCTION

FUNCTION ProcessPendingQueue()
    WHILE pendingQueue IS NOT EMPTY DO
        nextID = DEQUEUE_HIGHEST_PRIORITY(pendingQueue)
        IF interruptTable[nextID].priority > currentPriority THEN
            MasterInterruptHandler(nextID) // re-enter, may nest again
        ELSE
            // Not urgent enough yet; requeue and stop for now
            ENQUEUE(pendingQueue, nextID)
            BREAK
        END IF
    END WHILE
END FUNCTION

// Hardware trigger (called by CPU on IRQ line assertion)
ON_INTERRUPT_SIGNAL(interruptID)
    interruptTable[interruptID].pending = TRUE
    MasterInterruptHandler(interruptID)
END ON_INTERRUPT_SIGNAL

```

2. Scatter-Gather DMA Mechanism

Goal: transfer data between memory and I/O devices across multiple non-contiguous memory regions without continuous CPU involvement, by chaining descriptors that the DMA controller processes autonomously and signals back to the CPU only on completion or error.

```
STRUCTURE DMA_Descriptor
    sourceAddress  : Pointer
    destAddress    : Pointer
    length         : Integer
    nextDescriptor : Pointer // NULL if last in chain
    status         : {PENDING, IN_PROGRESS, COMPLETE, ERROR}
END STRUCTURE

GLOBAL descriptorList = EMPTY_LIST // linked list / ring of descriptors
GLOBAL activeDescriptor = NULL
GLOBAL DMA_ERROR_RETRY_LIMIT = 3

FUNCTION BuildDescriptorChain(memoryBlocks[])
    // memoryBlocks[] holds the non-contiguous (source, dest, length) tuples
    FOR EACH block IN memoryBlocks DO
        descriptor = NEW DMA_Descriptor
        descriptor.sourceAddress = block.source
        descriptor.destAddress   = block.dest
        descriptor.length       = block.length
        descriptor.status        = PENDING
        descriptor.nextDescriptor = NULL
        APPEND(descriptorList, descriptor)
    END FOR
    LINK_CONSECUTIVE(descriptorList) // set nextDescriptor pointers
    RETURN HEAD(descriptorList)
END FUNCTION

FUNCTION StartScatterGatherDMA(descriptorList)
    activeDescriptor = HEAD(descriptorList)
    IF activeDescriptor == NULL THEN
        RETURN ERROR_NO_DESCRIPTOR
    END IF
    ProgramDMAController(activeDescriptor)
    activeDescriptor.status = IN_PROGRESS
    DMA_Controller.Enable()
    RETURN SUCCESS
END FUNCTION

FUNCTION ProgramDMAController(descriptor)
    DMA_Controller.SetSource(descriptor.sourceAddress)
```

```

    DMA_Controller.SetDestination(descriptor.destAddress)
    DMA_Controller.SetLength(descriptor.length)
    DMA_Controller.SetInterruptOnComplete(TRUE)
END FUNCTION

// Fired by DMA hardware after each descriptor's transfer finishes
ON_DMA_INTERRUPT()
    IF DMA_Controller.TransferErrorFlag() THEN
        HandleTransferError(activeDescriptor)
        RETURN
    END IF

    activeDescriptor.status = COMPLETE
    nextDesc = activeDescriptor.nextDescriptor

    IF nextDesc != NULL THEN
        activeDescriptor = nextDesc
        ProgramDMAController(activeDescriptor)
        activeDescriptor.status = IN_PROGRESS
        DMA_Controller.Enable() // kick off next fragment, CPU stays free
    ELSE
        DMA_Controller.Disable()
        NotifyTransferComplete(descriptorList)
    END IF
END ON_DMA_INTERRUPT

FUNCTION HandleTransferError(descriptor)
    descriptor.status = ERROR
    descriptor.retryCount = descriptor.retryCount + 1

    IF descriptor.retryCount <= DMA_ERROR_RETRY_LIMIT THEN
        LogWarning("Retrying DMA descriptor", descriptor)
        ProgramDMAController(descriptor)
        descriptor.status = IN_PROGRESS
        DMA_Controller.Enable()
    ELSE
        LogError("DMA transfer aborted after max retries", descriptor)
        DMA_Controller.Disable()
        NotifyTransferFailed(descriptor)
    END IF
END FUNCTION

```

3. I/O Buffering Mechanism

Goal: absorb rate mismatches between producers and consumers of I/O data using circular buffers, watermark-based flow control, and explicit handling of buffer-full and buffer-empty conditions to prevent data loss.

```
STRUCTURE Buffer
    data[BUFFER_CAPACITY] : Byte
    head    : Integer // next write position
    tail    : Integer // next read position
    count   : Integer // bytes currently stored
END STRUCTURE

GLOBAL inputBuffer = NEW Buffer() // producer: device -> system
GLOBAL outputBuffer = NEW Buffer() // producer: system -> device
GLOBAL HIGH_WATERMARK = 0.85 * BUFFER_CAPACITY
GLOBAL LOW_WATERMARK = 0.15 * BUFFER_CAPACITY

FUNCTION BufferWrite(buffer, incomingData[], length)
    LOCK(buffer)
    bytesWritten = 0
    WHILE bytesWritten < length DO
        IF buffer.count == BUFFER_CAPACITY THEN
            // Buffer-full condition
            HandleBufferFull(buffer)
            IF buffer.count == BUFFER_CAPACITY THEN
                UNLOCK(buffer)
                RETURN bytesWritten // caller must retry/back off
            END IF
        END IF
        buffer.data[buffer.head] = incomingData[bytesWritten]
        buffer.head = (buffer.head + 1) MOD BUFFER_CAPACITY
        buffer.count = buffer.count + 1
        bytesWritten = bytesWritten + 1

        IF buffer.count >= HIGH_WATERMARK THEN
            SignalConsumer(buffer) // wake consumer early to avoid overflow
        END IF
    END WHILE
    UNLOCK(buffer)
    RETURN bytesWritten
END FUNCTION

FUNCTION BufferRead(buffer, maxLength)
    LOCK(buffer)
    IF buffer.count == 0 THEN
```

```

    // Buffer-empty condition
    HandleBufferEmpty(buffer)
    IF buffer.count == 0 THEN
        UNLOCK(buffer)
        RETURN EMPTY_RESULT
    END IF
END IF

readLength = MIN(maxLength, buffer.count)
outData[readLength]
FOR i = 0 TO readLength - 1 DO
    outData[i] = buffer.data[buffer.tail]
    buffer.tail = (buffer.tail + 1) MOD BUFFER_CAPACITY
    buffer.count = buffer.count - 1
END FOR

IF buffer.count <= LOW_WATERMARK THEN
    SignalProducer(buffer) // wake producer, room is available again
END IF
UNLOCK(buffer)
RETURN outData
END FUNCTION

FUNCTION HandleBufferFull(buffer)
    IF POLICY == BLOCK THEN
        WAIT_UNTIL(buffer.count < BUFFER_CAPACITY, TIMEOUT)
    ELSE IF POLICY == DROP_OLDEST THEN
        buffer.tail = (buffer.tail + 1) MOD BUFFER_CAPACITY
        buffer.count = buffer.count - 1
        LogWarning("Buffer overflow: oldest sample dropped")
    ELSE IF POLICY == ALLOCATE_OVERFLOW_BLOCK THEN
        ExpandBuffer(buffer, OVERFLOW_INCREMENT)
    END IF
END FUNCTION

FUNCTION HandleBufferEmpty(buffer)
    IF POLICY == BLOCK THEN
        WAIT_UNTIL(buffer.count > 0, TIMEOUT)
    ELSE
        LogInfo("No data available; consumer will poll again")
    END IF
END FUNCTION

```


4. Priority-Based Resource Allocation Mechanism

Goal: allocate shared communication resources (bandwidth, channels) among competing devices according to urgency and bandwidth requirements, while dynamically aging waiting devices' priority to prevent indefinite starvation of low-priority devices.

```
STRUCTURE Device
    id          : Integer
    basePriority  : Integer
    bandwidthNeeded : Integer
    waitTime     : Integer // time spent waiting for resource
    dynamicPriority : Integer
    state        : {IDLE, WAITING, ALLOCATED, TRANSMITTING}
END STRUCTURE

GLOBAL deviceList = [device1, device2, ..., deviceN]
GLOBAL availableBandwidth = TOTAL_SYSTEM_BANDWIDTH
GLOBAL AGING_FACTOR = 1 // priority boost per scheduling tick waited
GLOBAL MAX_WAIT_THRESHOLD = 50 // forces allocation to avoid starvation

FUNCTION UpdateDynamicPriorities()
    FOR EACH device IN deviceList DO
        IF device.state == WAITING THEN
            device.waitTime = device.waitTime + 1
            device.dynamicPriority = device.basePriority
                                + (device.waitTime * AGING_FACTOR)
        ELSE
            device.dynamicPriority = device.basePriority
        END IF
    END FOR
END FUNCTION

FUNCTION AllocateResources()
    UpdateDynamicPriorities()
    sortedDevices = SORT(deviceList, BY dynamicPriority DESCENDING)

    FOR EACH device IN sortedDevices DO
        IF device.state != WAITING THEN
            CONTINUE
        END IF

        // Starvation guard: force-allocate if waited too long
        forceAllocate = (device.waitTime >= MAX_WAIT_THRESHOLD)

        IF device.bandwidthNeeded <= availableBandwidth OR forceAllocate THEN
            grantedBandwidth = MIN(device.bandwidthNeeded, availableBandwidth)
```

```

        availableBandwidth = availableBandwidth - grantedBandwidth
        device.state = ALLOCATED
        device.waitTime = 0
        BeginTransmission(device, grantedBandwidth)
    END IF
    // else: device remains WAITING; its priority keeps aging
END FOR
END FUNCTION

FUNCTION BeginTransmission(device, bandwidth)
    device.state = TRANSMITTING
    device.allocatedBandwidth = bandwidth
    StartDeviceIO(device)
END FUNCTION

// Called when a device finishes sending/receiving its data
ON_TRANSMISSION_COMPLETE(device)
    availableBandwidth = availableBandwidth + device.allocatedBandwidth
    device.allocatedBandwidth = 0
    device.state = IDLE
    AllocateResources() // re-evaluate queue now that bandwidth freed up
END ON_TRANSMISSION_COMPLETE

// Scheduler tick, runs periodically (e.g. every system clock interrupt)
ON_SCHEDULER_TICK()
    AllocateResources()
END ON_SCHEDULER_TICK

```

5. Real-Time I/O Scheduling Mechanism

Goal: schedule data transfers from multiple peripherals to meet strict deadlines, using an Earliest-Deadline-First ordering informed by transfer duration, priority, and available bandwidth, while detecting and recovering from missed deadlines.

```
STRUCTURE IORequest
  deviceID      : Integer
  deadline      : Time    // absolute time by which transfer must finish
  duration      : Time    // estimated transfer time
  priority      : Integer
  bandwidthNeeded : Integer
  missedCount   : Integer
END STRUCTURE

GLOBAL requestQueue = EMPTY_LIST
GLOBAL availableBandwidth = TOTAL_SYSTEM_BANDWIDTH
GLOBAL currentTime = SystemClock()

FUNCTION ScheduleIORequests()
  currentTime = SystemClock()

  // Step 1: Rank by urgency -- Earliest-Deadline-First, tie-broken by priority
  feasible = []
  FOR EACH req IN requestQueue DO
    req.slack = req.deadline - currentTime - req.duration
    IF req.slack < 0 THEN
      HandleMissedDeadline(req)
    ELSE
      APPEND(feasible, req)
    END IF
  END FOR

  sortedRequests = SORT(feasible, BY deadline ASCENDING, THEN priority
DESCENDING)

  // Step 2: Admit requests while bandwidth and time budget allow
  FOR EACH req IN sortedRequests DO
    IF req.bandwidthNeeded <= availableBandwidth THEN
      availableBandwidth = availableBandwidth - req.bandwidthNeeded
      DispatchTransfer(req)
      REMOVE(requestQueue, req)
    END IF
    // else: leave in queue, retried on next scheduling pass
  END FOR
END FUNCTION
```

```

FUNCTION DispatchTransfer(req)
    req.startTime = SystemClock()
    StartDeviceTransfer(req.deviceID, req.bandwidthNeeded)
END FUNCTION

// Called by device driver / DMA layer when a transfer finishes
ON_TRANSFER_COMPLETE(req)
    availableBandwidth = availableBandwidth + req.bandwidthNeeded
    LogCompletion(req, SystemClock())
    ScheduleIORequests() // bandwidth freed: re-run scheduling pass
END ON_TRANSFER_COMPLETE

FUNCTION HandleMissedDeadline(req)
    req.missedCount = req.missedCount + 1
    LogWarning("Deadline missed for device", req.deviceID)

    // Reschedule with adjusted deadline and boosted priority
    req.deadline = currentTime + REASSIGNED_DEADLINE_MARGIN
    req.priority = req.priority + MISSED_DEADLINE_PRIORITY_BOOST
    APPEND(requestQueue, req)

    IF req.missedCount > MAX_ALLOWED_MISSES THEN
        RaiseSystemAlert("Persistent deadline failure", req.deviceID)
    END IF
END FUNCTION

// Periodic scheduling tick driven by the real-time clock
ON_TIMER_TICK()
    ScheduleIORequests()
END ON_TIMER_TICK

```